Université Hassan II de Casablanca

École Nationale Supérieure d'Arts et Métiers

ENSAM – Casablanca

Département Génie Informatique

1re année du cycle d'ingénieur

Intelligence Artificielle et Génie Informatique (IAGI)

Rapport de projet

Module : Python avancé

Application de gestion de stock d'un magasin informatique

Réalisé par Ennaji Aymen
Lakhtib Sami
Erraghay Mohamed Amine

Encadré par M. Mustapha Hain

Dépôt du projet

https://github.com/Sami-Lakhtib/Gestion-d-un-Magazin-Informatique

Année universitaire 2026/2027

Table des matières

1 Introduction **2**

2 Objectifs **2**

3 Outils utilisés **2**

4 Base de données **2**

4.1 Table `client` 2

4.2 Table `produit` 3

4.3 Table `vente` 3

5 Architecture **3**

6 Fonctionnalités **4**

7 Exemple : enregistrer une vente **4**

8 Lancement **4**

9 Limites **5**

10 Conclusion **5**

1. Introduction

Ce rapport présente une application de bureau destinée à la gestion de stock d'un magasin informatique. Elle permet d'enregistrer des clients et des produits, de suivre les quantités disponibles, d'enregistrer des ventes et de signaler les articles dont le stock devient trop bas.

Le sujet reste volontairement à l'échelle d'un projet de module. L'application fonctionne en local, avec une interface graphique et une base SQLite de trois tables. L'accent est mis sur une organisation du code lisible : l'affichage, les règles du magasin et les requêtes SQL sont séparés.

Le code source est disponible à l'adresse suivante :

`https://github.com/Sami-Lakhtib/Gestion-d-un-Magazin-Informatique`

2. Objectifs

Le travail répond à quatre objectifs :

— modéliser un stock simple avec les entités client, produit et vente ;

— mettre à jour automatiquement la quantité lors d'une vente ou d'un réapprovisionnement ;

— avertir l'utilisateur lorsqu'un produit atteint son seuil d'alerte ;

— structurer le programme en couches, afin que chaque partie ait un rôle précis.

3. Outils utilisés

Outil	Rôle
Python 3	Langage du programme
Tkinter	Interface graphique
SQLite	Base de données locale (`magazin.db`)

Ces trois outils appartiennent à la bibliothèque standard de Python. Le projet ne dépend d'aucun paquet à installer. L'interface est un tableau de bord : chaque bouton ouvre une fenêtre pour une action précise.

4. Base de données

La contrainte retenue pour le projet est de n'utiliser que trois tables.

4.1 Table `client`

Colonne	Type	Contrainte
`id_client`	entier	clé primaire
`nom`, `prenom`	texte	obligatoires
`telephone`	texte	12 chiffres, contrôlé par l'application
`email`	texte	unique et obligatoire
`adresse`	texte	obligatoire

4.2 Table `produit`

Colonne	Rôle
`id_produit`	Clé primaire
`nom`	Nom unique du produit
`categorie`	Famille (ordinateurs, périphériques, etc.)
`prix_achat`, `prix_vente`	Coût et prix de vente
`quantite_stock`	Quantité actuellement disponible
`seuil_alerte`	Seuil à partir duquel le produit est signalé
`description`	Court descriptif

4.3 Table `vente`

Colonne	Type	Contrainte
`id_vente`	entier	Clé primaire
`id_client`	entier	Clé étrangère vers `client`
`id_produit`	entier	Clé étrangère vers `produit`
`quantite`	entier	quantite vendu
`date_vente`	texte	date de vente
`prix_unitaire`	réel	prix d'une seul entite
`montant_total`	réel	montant total de la vente

Chaque ligne représente la vente d'un seul produit à un client. Elle mémorise la quantité, la date, le prix unitaire au moment de la vente et le montant total. Les colonnes `id_client` et `id_produit` relient la vente aux deux autres tables.

Le stock courant reste dans `produit`. Une vente le diminue ; un réapprovisionnement l'augmente. Les sorties déjà réalisées restent consultables dans `vente`.

Un produit est considéré en alerte lorsque :

$$quantitenstock \leq seuild'alerte.$$

5. Architecture

Le programme suit trois couches. L'interface ne contient pas de requête SQL, et les règles de gestion ne sont pas écrites dans les fenêtres.

Couche	Dossier	Responsabilité
Interface	`ui/`	Fenêtres, boutons, listes déroulantes, messages
Métier	`services/`	Validation, calcul du montant, mise à jour du stock, alertes
Données	`repositories/`	`INSERT`, `SELECT` et `UPDATE`

Les objets `Client`, `Produit` et `Vente` sont décrits dans `models/`. Le fichier `database.py` ouvre la connexion et crée les tables si elles n'existent pas. `main.py` se limite au démarrage : il crée la base, les services, puis le tableau de bord.

Toute action suit le même chemin :

$$fentre \rightarrow service \rightarrow repository \rightarrow \texttt{magazin.db}.$$

Cette séparation rend le projet plus simple à expliquer et à modifier. Ajouter un écran consiste à appeler un service déjà existant, sans réécrire le SQL dans l'interface.

6. Fonctionnalités

— **Ajouter un client.** Nom, prénom, téléphone, e-mail et adresse. Le téléphone doit compter 12 chiffres et l'e-mail doit contenir un arobase. Un e-mail déjà présent est refusé.

— **Ajouter un produit.** Nom, catégorie, prix, stock initial, seuil d'alerte et description. Le nom doit être unique.

— **Réapprovisionner.** L'utilisateur choisit un produit dans une liste déroulante et saisit la quantité à ajouter. Le stock est augmenté, puis les alertes du tableau de bord sont recalculées.

— **Enregistrer une vente.** Le produit est choisi dans la même liste. L'application vérifie le client, la quantité et le stock disponible, puis enregistre la vente et retire la quantité vendue.

— **Catalogue.** Tableau de tous les produits. Une ligne dont le stock est inférieur ou égal au seuil est affichée en rouge.

— **Liste des ventes.** Date, client, e-mail, produit, quantité, prix unitaire et total, obtenus par une jointure des trois tables.

— **Alertes du tableau de bord.** Liste des produits concernés, mise à jour au retour de chaque fenêtre.

7. Exemple : enregistrer une vente

1. L'utilisateur ouvre « Enregistrer une vente » et choisit un produit.
2. Il saisit l'e-mail du client et la quantité.
3. Le service recherche le produit et le client.
4. Si le produit ou le client est inconnu, si la quantité est invalide, ou si elle dépasse le stock, un message d'erreur s'affiche et rien n'est enregistré.
5. Sinon, une ligne est ajoutée dans `vente`, avec la date du jour et le montant $quantit \times prixdevente$.
6. Le stock du produit est diminué de la quantité vendue, puis la transaction est validée.
7. Au retour sur le tableau de bord, la liste des alertes est reconstruite.

Le prix unitaire est recopié dans la vente. Une modification ultérieure du prix du produit ne change donc pas le montant des ventes déjà passées.

8. Lancement

Depuis le dossier du projet :

```
python init_db.py
python main.py
```

`init_db.py` crée un jeu de démonstration : 6 clients, 10 produits et 6 ventes. L'un des produits, la clé USB, est volontairement sous son seuil afin de voir l'alerte dès le premier lancement. Relancer ce script efface les saisies et rétablit ces données.

Le code et ce rapport se trouvent sur le dépôt GitHub du projet :

https://github.com/Sami-Lakhtib/Gestion-d-un-Magazin-Informatique

9. Limites

L'application couvre le cycle utile d'un petit stock, dans le cadre fixé pour le module. Quelques choix en découlent :

- une vente concerne un seul produit à la fois ;
- il n'y a pas de compte utilisateur ni de rôle ;
- l'historique détaillé des entrées de stock n'est pas une table à part : seul le stock courant et les ventes sont conservés ;
- les prix saisis dans le formulaire produit sont des nombres entiers.

10. Conclusion

Le projet réalise une gestion de stock locale pour un magasin informatique : clients, catalogue, ventes, réapprovisionnement et alertes de seuil. La base reste limitée à trois tables, conformément au cadre du travail. La séparation entre interface, services et accès aux données permet de suivre clairement le traitement d'une vente, de la fenêtre jusqu'à SQLite.